



Universidad Autónoma de
Campeche.



FACULTAD DE INGENIERÍA.

INGENIERÍA EN TECNOLOGÍA EN SOFTWARE.

CONCURRENCIA Y PARALELISMO

Practica 01 - Sección Crítica

Sergio A Noh Puch

**San Francisco de Campeche, Campeche a
28 de agosto de 2026.**

Realizado por Cesar Enrique Uc Aguilar.

Matrícula: 67599.

Semestre y grupo: "5.A"

Condiciones de Carrera y Secciones Críticas en Programación Concurrente

La presente práctica tiene como objetivo implementar y analizar, mediante el lenguaje C y la biblioteca POSIX Threads (pthread), el comportamiento de dos hilos de ejecución que acceden de forma concurrente a una misma variable compartida. A partir de este ejercicio se busca comprender de manera práctica dos conceptos fundamentales de la programación concurrente: la condición de carrera (race condition) y la sección crítica (critical section), así como identificar, a nivel de código ensamblador, el origen físico del problema.

A partir del pseudocódigo proporcionado, se implementó un programa en C que declara una variable global llamada `global_counter`, inicializada en 20, y crea dos hilos mediante `pthread_create`. El primero, `thread_routine`, incrementa la variable 1000 veces dentro de un ciclo `for`; el segundo, `thread_routine_two`, la decrementa la misma cantidad de veces. Ambos hilos se ejecutan concurrentemente y, al finalizar, el programa espera su terminación con `pthread_join` antes de imprimir el valor final de `global_counter`. Dado que ambos hilos realizan la misma cantidad de operaciones inversas, el resultado matemáticamente esperado al finalizar la ejecución es que `global_counter` conserve su valor inicial de 20.

Una condición de carrera ocurre cuando el resultado de un programa depende del orden o del entrecruzamiento temporal en el que se ejecutan las instrucciones de dos o más hilos que acceden concurrentemente a un mismo recurso compartido, sin que exista un mecanismo que coordine dicho acceso. En estas circunstancias, el resultado final deja de ser determinista: puede variar entre una ejecución y otra, aun cuando el código fuente permanezca exactamente igual. El error conceptual más común al abordar este tema es asumir que una instrucción de alto nivel como `global_counter++` constituye una operación atómica, es decir, indivisible. En realidad, esta instrucción se traduce en el procesador en tres pasos independientes: leer el valor actual de la variable desde memoria hacia un registro del procesador, modificar ese valor dentro del registro sumando o restando 1, y escribir el nuevo valor de vuelta a la memoria. Como estos tres pasos no son atómicos, el sistema operativo puede interrumpir a un hilo en cualquier punto intermedio del proceso, típicamente mediante un cambio de contexto, y ceder el procesador al otro hilo. Si ambos hilos leen el valor de la variable antes de que el otro haya terminado de escribir su resultado, una de las dos actualizaciones se pierde, fenómeno conocido como *lost update* o actualización perdida.

Para ilustrar esto con un ejemplo concreto: supongamos que ambos hilos leen el valor 20 casi al mismo tiempo, antes de que ninguno haya escrito nada. El Hilo 1 suma 1 en su registro, obteniendo 21, pero aún no lo escribe en memoria. El Hilo 2, por su parte, resta 1 en su propio registro, obteniendo 19. Si el Hilo 1 escribe primero su resultado, la memoria pasa a valer 21; pero cuando el Hilo 2 escribe inmediatamente después, sobrescribe ese valor con 19, sin haber tomado en cuenta la actualización que hizo el Hilo 1. El resultado final termina siendo incorrecto respecto al valor esperado, y esa incorrección es exactamente la manifestación de la condición de carrera. Es importante señalar que ejecutar el programa varias veces y obtener repetidamente el valor correcto, 20, no demuestra la ausencia de una condición de carrera. Esto se debe a que el problema depende de la probabilidad de que el planificador del sistema operativo interrumpa a un hilo justo en el instante crítico entre la lectura y la escritura. Con solo 1000 iteraciones por hilo, esta

ventana de interrupción es extremadamente breve en comparación con la duración total del bucle, por lo que la manifestación visible del error puede no ocurrir en una ejecución particular, aunque la vulnerabilidad exista de forma latente en el código.

Al compilar el programa con la instrucción `gcc -S race_test.c`, se genera el archivo `race_test.s`, que contiene la representación en lenguaje ensamblador, en sintaxis AT&T y para arquitectura x86-64, del código fuente. Analizando específicamente los bloques correspondientes a `thread_routine` y `thread_routine_two`, sin optimizaciones del compilador, se observa que la instrucción `global_counter++` se traduce en tres líneas de ensamblador: primero, `movl global_counter(%rip), %eax`, que lee el valor actual de la variable desde memoria y lo coloca en el registro `%eax`; después, `addl $1, %eax`, que suma 1 al contenido de ese registro; y finalmente, `movl %eax, global_counter(%rip)`, que escribe el resultado de vuelta en memoria. De manera equivalente, la instrucción `global_counter--` genera el mismo patrón de lectura y escritura, con la única diferencia de que en el paso intermedio se usa `subl $1, %eax` en lugar de `addl`, restando 1 en vez de sumarlo.

La instrucción `movl`, que significa *move long*, transfiere un valor de 32 bits entre memoria y un registro, o entre registros, y aparece dos veces por cada operación: una para leer el valor original desde memoria hacia el registro `%eax`, y otra para devolver el resultado modificado desde el registro hacia la memoria. La instrucción `addl` realiza una suma de 32 bits, sumando la constante 1 al contenido del registro `%eax`, lo cual corresponde al operador `++` del código en C. La instrucción `subl`, por su parte, realiza una resta de 32 bits, restando 1 al contenido del registro `%eax`, correspondiente al operador `--`. Este análisis constituye la evidencia concreta, a nivel de arquitectura de procesador, de que la instrucción en C aparentemente simple `global_counter++` no es atómica: se descompone en tres instrucciones de máquina separadas, cada una de las cuales puede ser interrumpida por el planificador del sistema operativo para ceder el turno de ejecución a otro hilo. Es precisamente en el intervalo entre estas instrucciones donde puede producirse la condición de carrera descrita anteriormente.

Se denomina sección crítica al fragmento de código de un programa concurrente en el que se accede a uno o más recursos compartidos, como variables, estructuras de datos, archivos o dispositivos, de tal manera que su ejecución simultánea por parte de dos o más hilos puede provocar inconsistencias en el estado del recurso. En el programa analizado, la sección crítica corresponde precisamente a las líneas `global_counter++` y `global_counter--` dentro de cada rutina de hilo, ya que ambas acceden y modifican la misma variable global sin ningún tipo de exclusión mutua. Para garantizar la correcta ejecución de un programa concurrente, toda sección crítica debe satisfacer tres propiedades fundamentales. La primera es la exclusión mutua, que establece que, en un momento dado, únicamente un hilo puede estar ejecutando su sección crítica sobre el recurso compartido. La segunda es el progreso, que exige que, si ningún hilo se encuentra dentro de la sección crítica, aquellos hilos que deseen entrar no deben ser bloqueados indefinidamente por causas ajenas a la necesidad real de exclusión. La tercera es la espera acotada, que establece que debe existir un límite en la cantidad de veces que otros hilos pueden entrar a la sección crítica antes de que un hilo en espera obtenga su turno, evitando así la inanición o *starvation*.

La solución estándar en POSIX Threads para proteger una sección crítica es el uso de un mutex, abreviatura de mutual exclusion lock, una estructura que actúa como un candado. Antes de ingresar a la sección crítica,

el hilo debe adquirir el mutex mediante la función `pthread_mutex_lock`; si otro hilo ya lo posee, el hilo solicitante queda bloqueado hasta que el mutex sea liberado. Una vez finalizada la operación sobre el recurso compartido, el hilo libera el mutex mediante `pthread_mutex_unlock`, permitiendo que otro hilo en espera pueda ingresar. En el código de esta práctica se declaró la variable mutex de tipo `pthread_mutex_t`, la cual debe emplearse envolviendo las operaciones `global_counter++` y `global_counter--` con este par de funciones, de manera que cada hilo llame a `pthread_mutex_lock` antes de modificar la variable y a `pthread_mutex_unlock` inmediatamente después. Esto elimina por completo la posibilidad de una condición de carrera, forzando que las tres instrucciones ensambladoras, `movl`, `addl` o `subl`, y `movl`, se ejecuten de manera atómica desde la perspectiva de los demás hilos.

En conclusión, el ejercicio permitió comprobar que, aunque a nivel del lenguaje C una operación como `global_counter++` parece una instrucción atómica y trivial, en realidad se traduce en el ensamblador en una secuencia de tres instrucciones independientes, correspondientes a la lectura, la modificación y la escritura, susceptibles de ser interrumpidas por el planificador del sistema operativo. Esta característica es la causa raíz de las condiciones de carrera en programación concurrente, y su solución requiere delimitar explícitamente las secciones críticas del programa y protegerlas mediante mecanismos de sincronización como los mutex, garantizando así la exclusión mutua y la consistencia del estado compartido entre los hilos.

The screenshot shows the OnlineGDB interface with a C program that uses pthreads to demonstrate a race condition. The program declares a global counter, creates two threads that decrement the counter, and prints the initial and final values. The output shows both initial and final values as 20, indicating a race condition where the counter was not properly updated.

```

1  }
2  return NULL;
3  }
4
5  void *thread_routine_two(void *arg) {
6      for (size_t i = 0; i < 1000; i++) {
7          global_counter--;
8      }
9      return NULL;
10 }
11
12 int main() {
13     pthread_t t1, t2;
14
15     printf("Valor inicial de global_counter: %d\n", global_counter);
16
17     pthread_create(&t1, NULL, thread_routine, NULL);
18     pthread_create(&t2, NULL, thread_routine_two, NULL);
19
20     pthread_join(t1, NULL);
21     pthread_join(t2, NULL);
22
23     printf("Valor final de global_counter: %d\n", global_counter);
24
25     return 0;
26 }
27

```

input

```

Valor inicial de global_counter: 20
Valor final de global_counter: 20
...Program finished with exit code 0
Press ENTER to exit console.

```

